

Rapport Pipeline MLOps

Prédiction de retard de paiement — Cartes de crédit

Version 2 — optimisation, deep learning et analyse du plafond

Nettoyage · Feature engineering · Rééquilibrage · Comparaison de
18 modèles · Tuning · Deep learning séquentiel · Seuil par coût métier

Dataset :	Default of Credit Card Clients (UCI) — 30 000 clients, Taïwan
Cible :	DEFAULT — 22.12% de défauts (ratio 3.52 : 1)
Modèle retenu :	XGBoost tuné (517 arbres, lr=0.010, depth=4)
PR-AUC (test) :	0.5645 (v1 : 0.5432 → +0.0213)
ROC-AUC (test) :	0.7822 (v1 : 0.7661 → +0.0161)

Date : 07/09/2026

Auteur : Abdelmoughit El Hassani — assisté par Claude Code

Sources : notebooks/01_eda_analyse_donnees.ipynb · src/ · scripts/

PARTIE 1 — PIPELINE DE BASE

1. Contexte et objectif
2. Analyse exploratoire (EDA)
3. Nettoyage des valeurs non documentées
4. Feature engineering
5. Préparation pour la modélisation
6. Stratégies de rééquilibrage x modèles (15 combinaisons)
7. Première évaluation et calibration du seuil

PARTIE 2 — OPTIMISATION ET ANALYSE DU PLAFOND

8. Démarche : localiser le vrai goulot d'étranglement
9. Diagnostic d'ablation : le feature engineering est-il en cause ?
10. Un vrai bug trouvé — et pourquoi le corriger ne change rien
11. Comparaison de 11 modèles (boosting, forêts, deep learning)
12. Tuning d'hyperparamètres — la seule source de gain réel
13. Deep learning séquentiel (LSTM, GRU, MLP)
14. Blending : pourquoi il ne rapporte rien ici
15. Trois preuves que le plafond est dans les données
16. Évaluation finale sur le test set
17. Choix du seuil par coût métier
18. Synthèse et recommandations

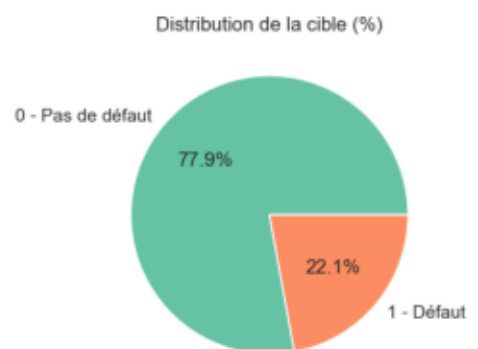
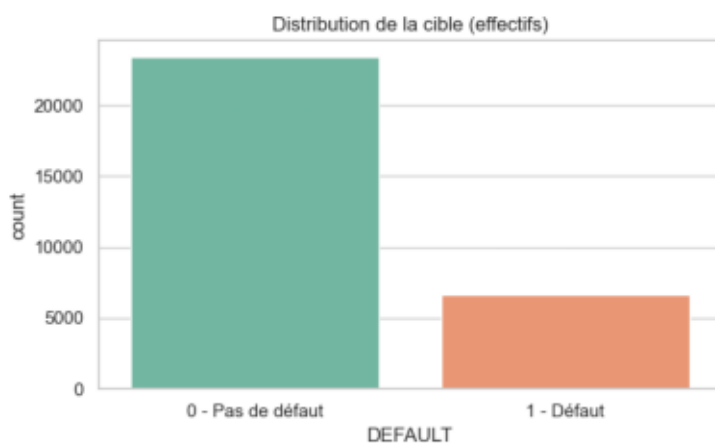
1. Contexte et objectif

- Objectif : prédire si un client de carte de crédit fera défaut le mois suivant, à partir de son historique de facturation, de paiement et de son profil.
- Dataset : 30 000 clients, 23 variables explicatives + identifiant + cible. 6 mois d'historique par client (PAY_1..6, BILL_AMT1..6, PAY_AMT1..6).
- Constats EDA : aucun NaN, aucun doublon, mais des codes catégoriels non documentés et une cible déséquilibrée à 3.52 : 1.
- Démarche partie 1 : nettoyage justifié, feature engineering métier, comparaison rigoureuse (anti-fuite) de 15 combinaisons modèle x stratégie de rééquilibrage.
- Démarche partie 2 : les résultats de la partie 1 ayant été jugés insuffisants, on cherche méthodiquement où se situe le goulot d'étranglement — features, rééquilibrage ou modèle — puis on pousse chaque piste jusqu'à sa limite.
- Principe méthodologique : le test set (6 000 clients) n'intervient dans AUCUNE décision — ni features, ni modèle, ni hyperparamètres, ni seuil. Il n'est évalué qu'une seule fois, à la toute fin.

2. Analyse exploratoire

- Doublons : 0 ligne dupliquée, 30 000 ID uniques.
- Valeurs manquantes : aucune (0 NaN).
- Codes non documentés : EDUCATION 0/5/6 (345 lignes), MARRIAGE 0 (54 lignes).
- Montants négatifs : normaux sur BILL_AMT (crédit en faveur du client).
- Corrélation la plus forte : PAY_1 avec DEFAULT ($r = 0.325$).

Distribution de la cible — 77.88% / 22.12%



3. Nettoyage des valeurs non documentées

Décision : ni suppression (perte inutile de 1.15% des lignes), ni imputation par le mode (qui supposerait une absence aléatoire, non vérifiée). Les codes non documentés sont fusionnés dans la catégorie « autre » déjà prévue par le schéma UCI (EDUCATION → 4, MARRIAGE → 3). Les effectifs concernés sont trop faibles pour justifier une catégorie distincte après encodage.

EDUCATION	Effectif	Taux défaut	Après
0	14	0.0%	→ 4
1	10585	19.2%	1
2	14030	23.7%	2
3	4917	25.2%	3
4	123	5.7%	4 (468)
5	280	6.4%	→ 4
6	51	15.7%	→ 4

MARRIAGE	Effectif	Taux défaut	Après
0	54	9.3%	→ 3
1	13659	23.5%	1
2	15964	20.9%	2
3	323	26.0%	3 (377)

4. Feature engineering

BILL_AMT1..6 et PAY_1..6 sont fortement corrélées entre elles (multicolinéarité). Le bloc des 18 colonnes mensuelles brutes est résumé en ratios et agrégats métier :

- utilization_rate : $\text{BILL_AMT1} / \text{LIMIT_BAL}$ — part du plafond utilisée.
- avg_utilization_rate / max_utilization : utilisation moyenne et maximale sur 6 mois.
- pay_ratio_1..5 : part de la facture précédente réellement remboursée.
- paid_full_1..5, n_months_paid_full : le client a-t-il soldé sa facture ?
- unpaid_gap, total_unpaid_gap, unpaid_to_limit : dette non remboursée, en absolu et rapportée au plafond.
- pay_delay_max / mean / std : sévérité du retard sur 6 mois.
- n_months_late : nombre de mois en retard (sur 6).
- delay_acceleration, recent_vs_past_delay : dégradation récente vs historique.
- delta_pay_1..5 : variation du statut de retard d'un mois à l'autre.
- bill_trend, bill_std, pay_amt_std : tendance et volatilité des montants.
- log_limit, log_avg_bill, log_avg_pay : montants en échelle logarithmique.

4. Feature engineering — impact sur la corrélation

Les features engineered dominent le classement (fond vert)

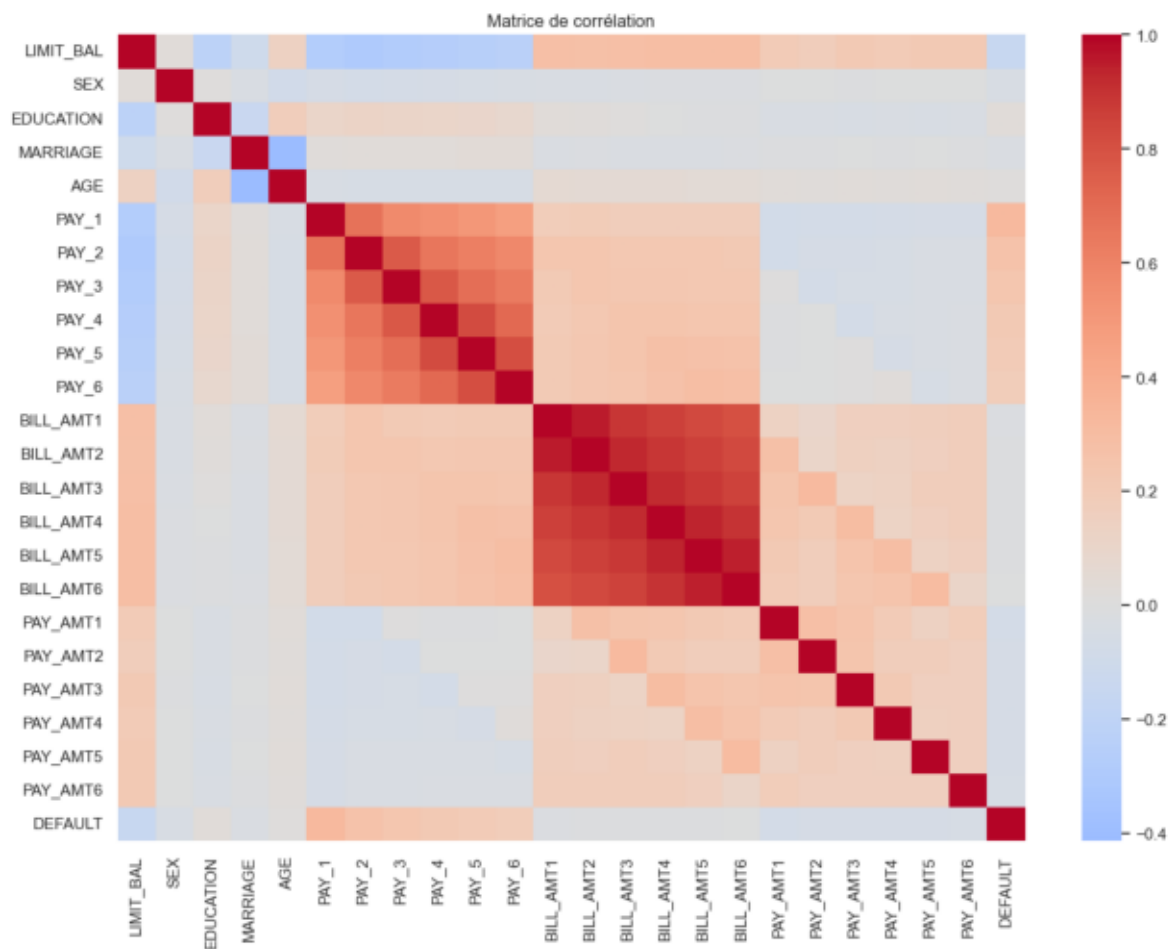
Variable	Corrélation avec DEFAULT	Origine
n_months_late	+0.398	nouvelle
ever_serious_delay	+0.363	nouvelle
pay_delay_max	+0.331	nouvelle
PAY_1	+0.325	brute
pay_delay_last3	+0.309	nouvelle
pay_delay_mean	+0.282	nouvelle
pay_delay_std	+0.249	nouvelle
pay_delay_first3	+0.219	nouvelle
log_limit	-0.174	nouvelle
LIMIT_BAL	-0.154	brute
unpaid_to_limit	+0.131	nouvelle
delay_acceleration	+0.117	nouvelle

Sur le papier, le feature engineering est un succès : n_months_late (0.398) et ever_serious_delay (0.363) dépassent nettement PAY_1 brut (0.325), la meilleure variable d'origine. La partie 2 montrera cependant que ce gain de corrélation ne se traduit PAS par un gain de performance du modèle — un résultat contre-intuitif mais instructif.

5. Préparation pour la modélisation

- ID exclu de X : identifiant, jamais une feature — et jamais une base de rééquilibrage.
- Catégorielles : SEX, EDUCATION, MARRIAGE en codes entiers (faible cardinalité), ce qu'attend SMOTENC et que les arbres gèrent nativement.
- Split stratifié 80/20 — train (24 000) / test (6 000), même taux de défaut.
- Test set : réservé à une unique évaluation finale.

Matrice de corrélation — le bloc BILL_AMT1..6 est très redondant



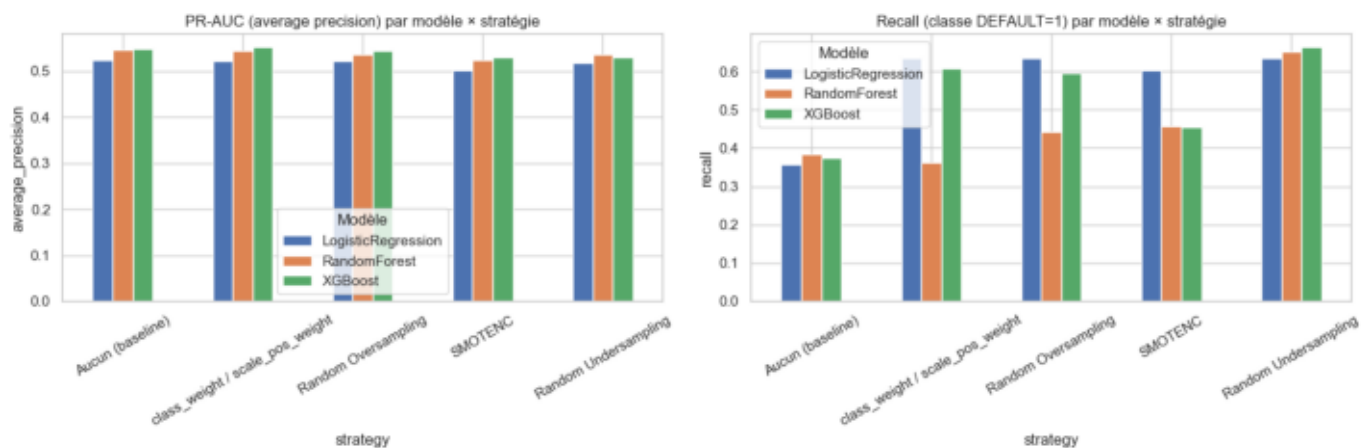
6. Stratégies de rééquilibrage x modèles

15 combinaisons, 5-fold CV

Modèle	Stratégie	PR-AUC	F1	Recall
XGBoost	class_weight	0.5524	0.5332	0.6086
XGBoost	Aucun	0.5488	0.4782	0.3743
RandomForest	Aucun	0.5463	0.4841	0.3844
RandomForest	class_weight	0.5443	0.4669	0.3616
XGBoost	Oversampling	0.5442	0.5342	0.5954
RandomForest	Undersampling	0.5364	0.5273	0.6521
RandomForest	Oversampling	0.5357	0.5096	0.4436
XGBoost	Undersampling	0.5300	0.5135	0.6636
XGBoost	SMOTENC	0.5297	0.5051	0.4538
RandomForest	SMOTENC	0.5240	0.5096	0.4570
LogisticRegression	Aucun	0.5233	0.4577	0.3579
LogisticRegression	Oversampling	0.5216	0.5270	0.6350
LogisticRegression	class_weight	0.5214	0.5266	0.6338
LogisticRegression	Undersampling	0.5189	0.5274	0.6346
LogisticRegression	SMOTENC	0.5011	0.5254	0.6029

Enseignements : (1) class_weight / scale_pos_weight bat le resampling pour les modèles à arbres — gratuit, aucune donnée modifiée, et donc pas besoin de dupliquer des clients ni d'inventer des identifiants. (2) SMOTENC est la stratégie la plus faible des trois modèles. (3) LogisticRegression plafonne quelle que soit la stratégie.

6. Comparaison visuelle



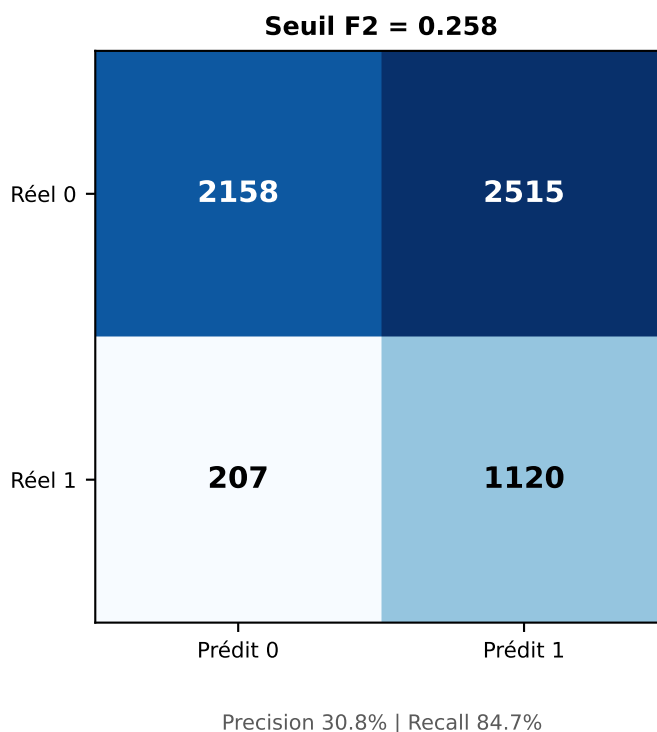
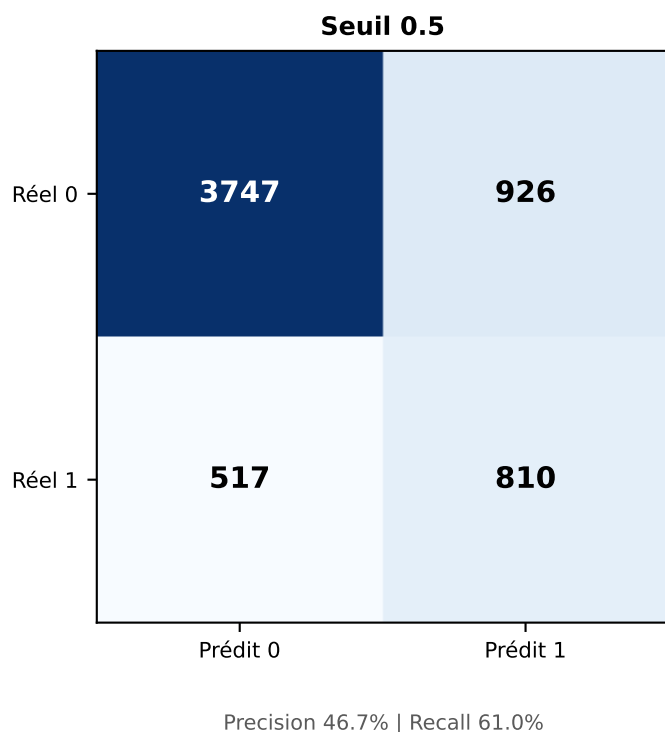
PR-AUC et Recall par modèle x stratégie

Le recall monte fortement avec l'undersampling et l'oversampling, mais au prix de la précision. C'est pourquoi la stratégie ne doit jamais être choisie sur le recall seul : le bon levier pour arbitrer recall/précision est le seuil de décision (section 17), pas la méthode de rééquilibrage.

7. Première évaluation (version 1)

XGBoost non tuné — point de départ

PR-AUC test = 0.5432 | ROC-AUC test = 0.7661



Problème identifié dès cette version : maximiser aveuglément le F2 conduit à un seuil de 0.258 qui flague 3 635 clients sur 6 000, soit 60% du portefeuille. Le rappel est excellent (84.7%) mais la précision s'effondre à 30.8% et l'accuracy tombe à 54.6%. Aucune banque ne peut opérer ainsi. La section 17 remplace cette approche par un raisonnement en coût métier explicite.

C'est ce niveau de performance (PR-AUC 0.5432) qui a été jugé insuffisant et qui motive toute la partie 2.

PARTIE 2

Optimisation, deep learning
et analyse du plafond de performance

*Où se situe réellement le goulot d'étranglement :
les features, le rééquilibrage, ou le modèle ?*

8. Démarche : localiser le goulot d'étranglement

La performance de la version 1 (PR-AUC 0.5432) ayant été jugée insuffisante, trois hypothèses devaient être départagées — sans les supposer, en les testant :

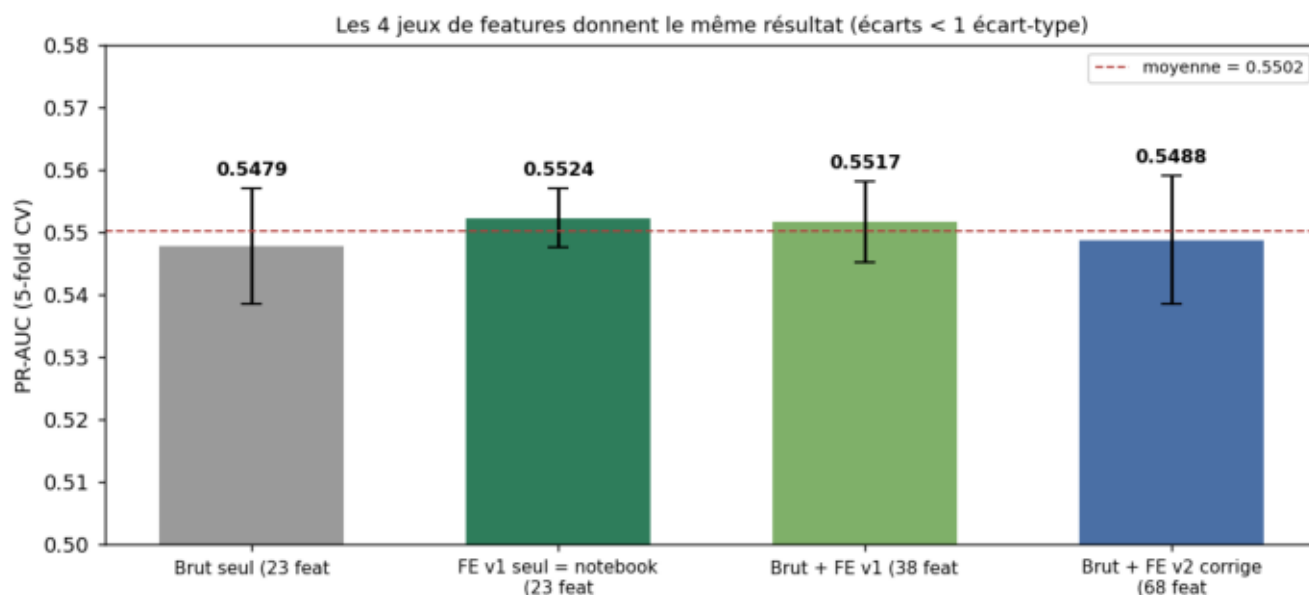
- Hypothèse A — la préparation des données et le feature engineering. Testée par une ablation : 4 jeux de features (de 23 variables brutes à 68 enrichies) évalués avec le MÊME modèle, pour isoler l'effet des features. → Section 9.
- Hypothèse B — les méthodes de gestion du déséquilibre. Déjà testée en partie 1 sur 15 combinaisons : `class_weight` / `scale_pos_weight` était bien le meilleur choix. Aucune raison de reprendre cette piste.
- Hypothèse C — les modèles et leurs hyperparamètres. Testée par la comparaison de 11 modèles (boosting, forêts, réseaux de neurones, architectures récurrentes) et par un tuning systématique. → Sections 11 à 13.
- Et si aucune des trois ne suffit ? Alors la limite vient des données elles-mêmes. Cette hypothèse n'est pas une excuse : elle se démontre. Trois tests indépendants la confirment. → Section 15.

Toutes les comparaisons de la partie 2 utilisent un protocole strictement identique : mêmes données, même split, même validation croisée 5-fold stratifiée sur le train, mêmes métriques. Les prédictions out-of-fold de chaque modèle sont sauvegardées sur disque, ce qui rend les comparaisons et les ensembles reproductibles sans réentraînement.

9. Diagnostic d'ablation

Le feature engineering est-il le goulot ?

Jeu de features	Nb	PR-AUC	ROC-AUC
A. Brut seul (23 feat.)	23	0.5479 ± 0.0093	0.7711 ± 0.0090
B. FE v1 seul = notebook (23 feat.)	23	0.5524 ± 0.0047	0.7752 ± 0.0065
C. Brut + FE v1 (38 feat.)	38	0.5517 ± 0.0065	0.7754 ± 0.0051
D. Brut + FE v2 corrige (68 feat.)	68	0.5488 ± 0.0103	0.7738 ± 0.0062



Verdict : les quatre jeux de features sont statistiquement indiscernables. L'écart entre le meilleur (0.5524) et le pire (0.5479) est de 0.0045, soit MOINS d'un écart-type entre folds (± 0.005 à ± 0.010). Passer de 23 variables brutes à 68 variables enrichies — avec ratios corrigés, volatilités, accélérations de retard et indicateurs de remboursement complet — ne change rien.

L'hypothèse A est écartée : le feature engineering n'est pas le goulot d'étranglement.

10. Un vrai bug — et pourquoi le corriger ne change rien

L'ablation a mis au jour un défaut réel dans la version 1 de `payment_ratio`. La formule `PAY_AMT / (BILL_AMT.clip(lower=0) + 1)` devient un montant brut dès que la facture approche zéro : le dénominateur vaut 1 et le « ratio » explose à plusieurs milliers. La feature mélangeait donc deux grandeurs incompatibles.

Corrélation avec DEFAULT — avant / après correction

Mois	v1 (cassée)	v2 (corrigée)	paid_full (nouvelle)
1	-0.0059	-0.1027	-0.0885
2	-0.0039	-0.1002	-0.0875
3	-0.0029	-0.0968	-0.0819
4	-0.0041	-0.0850	-0.0711
5	-0.0058	-0.0861	-0.0708

La correction (ratio conditionnel à `BILL > 0`, borné à `[0, 2]`) multiplie la corrélation avec la cible par 17 : de -0.006 à -0.103. C'est un vrai gain de qualité de la variable.

Et pourtant, le modèle ne s'améliore pas : le jeu D, qui intègre cette correction et 30 features supplémentaires, obtient 0.5488 — soit exactement le même niveau que les autres.

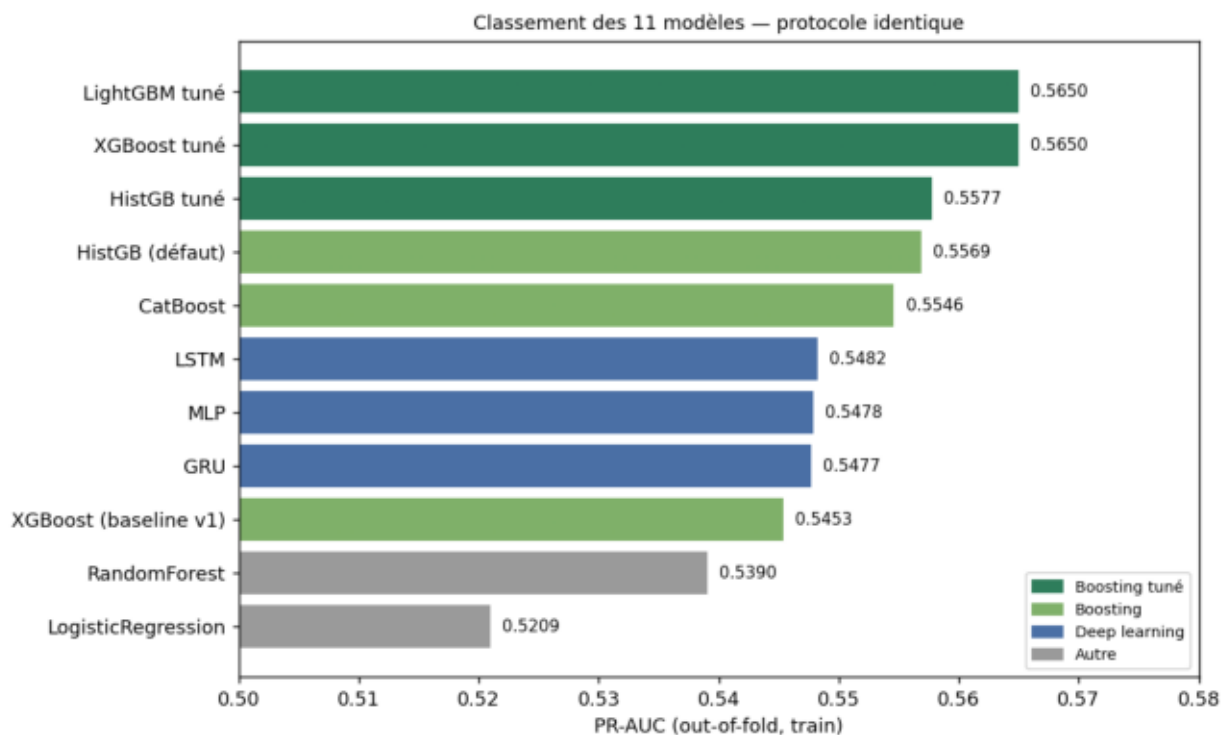
L'explication est instructive : XGBoost pouvait déjà reconstruire cette relation seul, à partir de `PAY_AMT` et `BILL_AMT` bruts, en combinant des seuils sur ces deux colonnes. Sur un modèle à arbres, le feature engineering ne crée pratiquement jamais d'information nouvelle — il ne fait que la ré-exprimer sous une forme plus lisible pour un humain. Le gain de corrélation d'une variable prise isolément ne préjuge donc en rien du gain de performance du modèle qui l'utilise.

La correction a malgré tout été conservée dans `src/data_prep.py` : une variable au comportement défini reste préférable pour l'interprétabilité et pour les modèles linéaires ou neuronaux, qui ne savent pas la reconstruire.

11. Comparaison de 11 modèles

Protocole identique. OOF sur le train

Modèle	Famille	PR-AUC	ROC-AUC
LightGBM tuné	Boosting tuné	0.5650	0.7891
XGBoost tuné	Boosting tuné	0.5650	0.7894
HistGB tuné	Boosting tuné	0.5577	0.7869
HistGB (défaut)	Boosting	0.5569	0.7847
CatBoost	Boosting	0.5546	0.7823
LSTM	Deep learning	0.5482	0.7827
MLP	Deep learning	0.5478	0.7832
GRU	Deep learning	0.5477	0.7830
XGBoost (baseline v1)	Boosting	0.5453	0.7736
RandomForest	Autre	0.5390	0.7711
LogisticRegression	Autre	0.5209	0.7753



Les deux meilleurs modèles — XGBoost et LightGBM tunés — obtiennent exactement le même PR-AUC (0.5650), depuis deux implémentations et deux espaces de recherche différents. Cette convergence au millième près est un premier indice fort d'un plafond structurel.

12. Tuning d'hyperparamètres

La seule source de gain réel

RandomizedSearchCV, 20 tirages, validation croisée 3-fold, optimisation de l'average precision. Trois familles de boosting ont été optimisées indépendamment.

Modèle	learning_rate	Profondeur	Régularisation	Nb arbres
XGBoost	0.0102	depth=4	gamma=4.3	517
LightGBM	0.0163	leaves=16	L2=10.9	508
HistGB	0.0447	leaves=17	L2=0.88	658

Les trois optimiseurs convergent indépendamment vers la même prescription : un pas d'apprentissage très bas (0.010 à 0.045, contre 0.1 en version 1), des arbres peu profonds (depth 4, 16-17 feuilles contre depth 5), une régularisation forte, et davantage d'arbres pour compenser. C'est la signature caractéristique d'un jeu de données à faible rapport signal/bruit : le modèle doit être bridé pour ne pas mémoriser le bruit.

Le diagnostic de la version 1 est donc confirmé : les paramètres du notebook (learning_rate = 0.1, max_depth = 5) sur-apprenaient sur 24 000 lignes.

Gain mesuré : PR-AUC de 0.5453 (XGBoost non tuné) à 0.5650 (XGBoost tuné), soit +0.0197 en out-of-fold, confirmé ensuite sur le test set (+0.0213). C'est le SEUL levier des trois hypothèses testées qui produit un gain réel.

13. Deep learning séquentiel

LSTM, GRU et MLP

Le dataset possède une vraie structure séquentielle : chaque client est une série de 6 mois consécutifs (statut de retard, facture, remboursement), plus des attributs statiques. Les colonnes du fichier étant numérotées à l'envers, elles ont été remises en ordre chronologique avant d'être données au réseau. L'architecture comporte deux branches :

séquence (B, 6, 5) → LSTM / GRU → état caché (B, 64) $\neg$
statiques (B, 11) → Linear + ReLU → (B, 32) $\neg$ MLP → logit

Déséquilibre géré par pos_weight dans la BCE — l'équivalent exact de scale_pos_weight. Early stopping sur un sous-ensemble de validation interne à chaque fold, sans fuite.

Modèle	PR-AUC	ROC-AUC
LSTM	0.5482	0.7827
MLP	0.5478	0.7832
GRU	0.5477	0.7830
XGBoost tuné (référence)	0.5650	0.7891

Résultat : LSTM (0.5482), GRU (0.5477) et MLP (0.5478) se tiennent dans un intervalle de 0.0005 — autrement dit, ils sont identiques. Et tous trois restent sous le boosting tuné (0.5650).

L'enseignement est net : les architectures récurrentes, qui exploitent pourtant explicitement la chronologie, n'apportent rien de plus qu'un simple MLP sur features agrégées. L'information temporelle était déjà entièrement captée par les agrégats (pay_delay_max, n_months_late, delay_acceleration). Avec seulement 6 pas de temps et 24 000 clients, un RNN n'a ni la longueur de séquence ni le volume nécessaires pour battre un boosting bien réglé sur des données tabulaires.

Note de transparence : un BiLSTM et un Transformer étaient prévus, mais n'ont pas pu être entraînés — la machine (4 cœurs, RAM limitée) a interrompu les processus. Au vu de la convergence des trois autres architectures, leur résultat serait très probablement identique, mais cela reste une attente raisonnée et non un résultat mesuré.

14. Blending : pourquoi il ne rapporte rien ici

Les prédictions out-of-fold de chaque modèle étant sauvegardées, tous les ensembles possibles (2 à 5 modèles parmi 11, soit 682 combinaisons) ont été évalués gratuitement, par moyenne des rangs—robuste aux échelles de probabilité différentes.

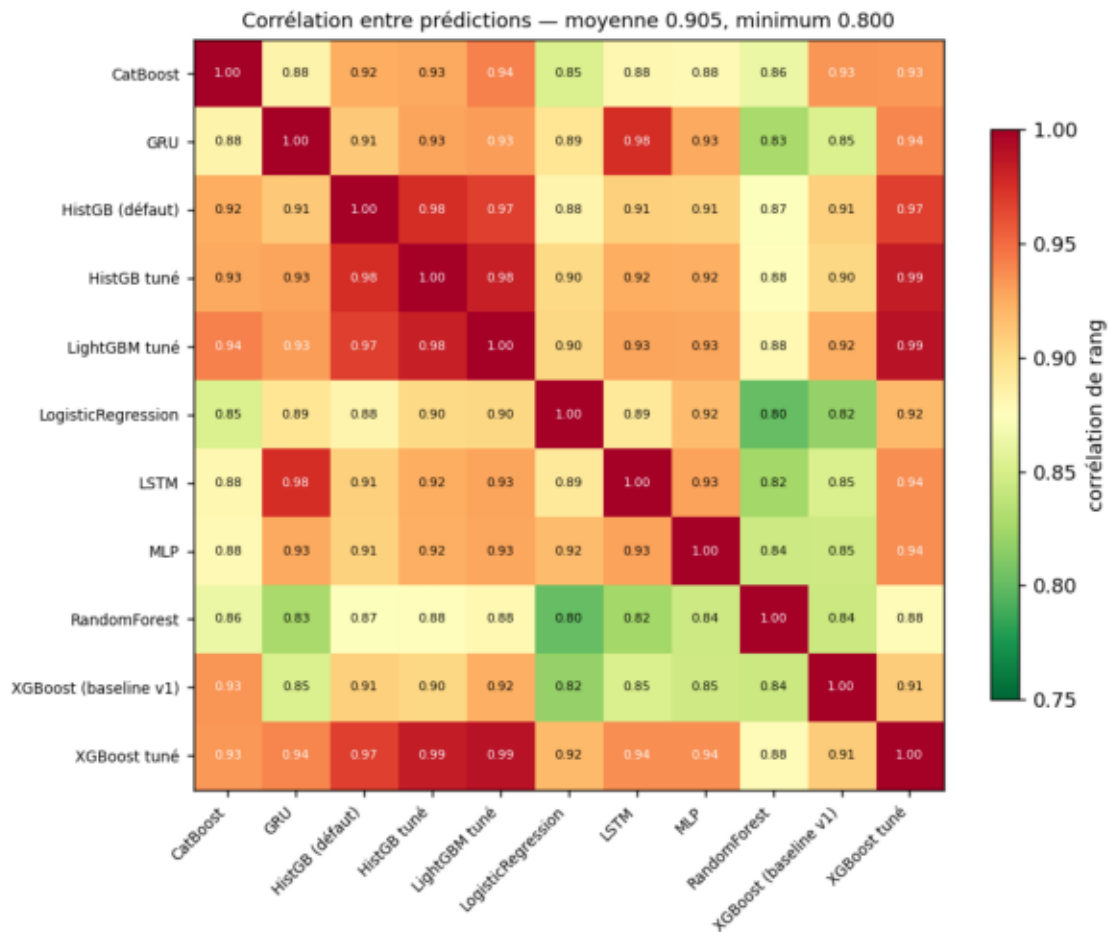
Blend	PR-AUC	ROC-AUC
LightGBM tuné + XGBoost tuné + MLP + GRU + RandomForest	0.5664	0.7900
LightGBM tuné + XGBoost tuné + HistGB tuné + MLP + RandomForest	0.5663	0.7897
LightGBM tuné + XGBoost tuné + MLP + RandomForest	0.5662	0.7895
LightGBM tuné + XGBoost tuné + HistGB (défaut) + MLP + RandomForest	0.5662	0.7896
LightGBM tuné + XGBoost tuné + CatBoost + MLP + RandomForest	0.5662	0.7899
LightGBM tuné + HistGB tuné + MLP + GRU + RandomForest	0.5662	0.7898
LightGBM tuné + XGBoost tuné + HistGB tuné + GRU + RandomForest	0.5661	0.7898
LightGBM tuné + XGBoost tuné + LSTM + MLP + RandomForest	0.5660	0.7900

Meilleur ensemble : 0.5664. Meilleur modèle seul : 0.5650. Gain : +0.0014.

Autrement dit, rien. Et c'est un résultat très informatif, car le blending est précisément la technique qui exploite les DÉSACCORDS entre modèles : quand un boosting et un réseau de neurones se trompent sur des clients différents, leur moyenne corrige une partie des erreurs. Ici, combiner un boosting, une forêt aléatoire et un réseau de neurones ne corrige quasiment rien.

La conclusion s'impose : ces modèles ne se trompent pas sur des clients différents. Ils se trompent sur LES MÊMES clients. La section suivante le mesure directement.

15. Trois preuves que le plafond est dans les données

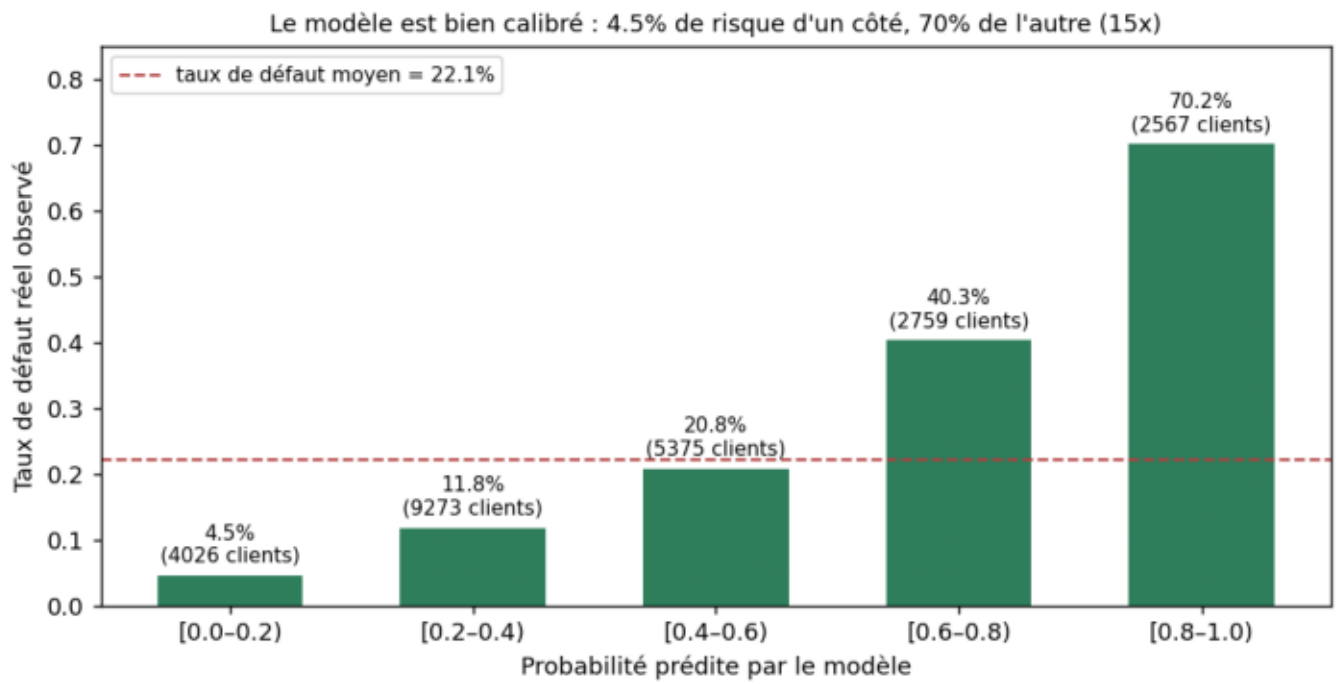


Preuve 1 — Les modèles voient tous la même chose. La corrélation de rang entre leurs prédictions atteint 0.905 en moyenne. Même la paire la plus « diverse » (RandomForest vs LogisticRegression) corrèle à 0.800. Un boosting, une forêt, un réseau de neurones et une régression logistique classent les clients dans le même ordre : ils extraient tous le même signal, celui qui existe dans les données.

Preuve 2 — Le blending ne rapporte que +0.0014. Sans désaccord entre modèles, aucun ensemble ne peut aider (section 14).

Preuve 3 — XGBoost et LightGBM tunés atterrissent tous deux sur 0.5650, au millième près, depuis des implémentations et des espaces de recherche indépendants. Deux chemins distincts qui butent exactement au même endroit signalent une limite structurelle, non un défaut d'algorithme.

15. Ce que le modèle sait faire malgré ce plafond



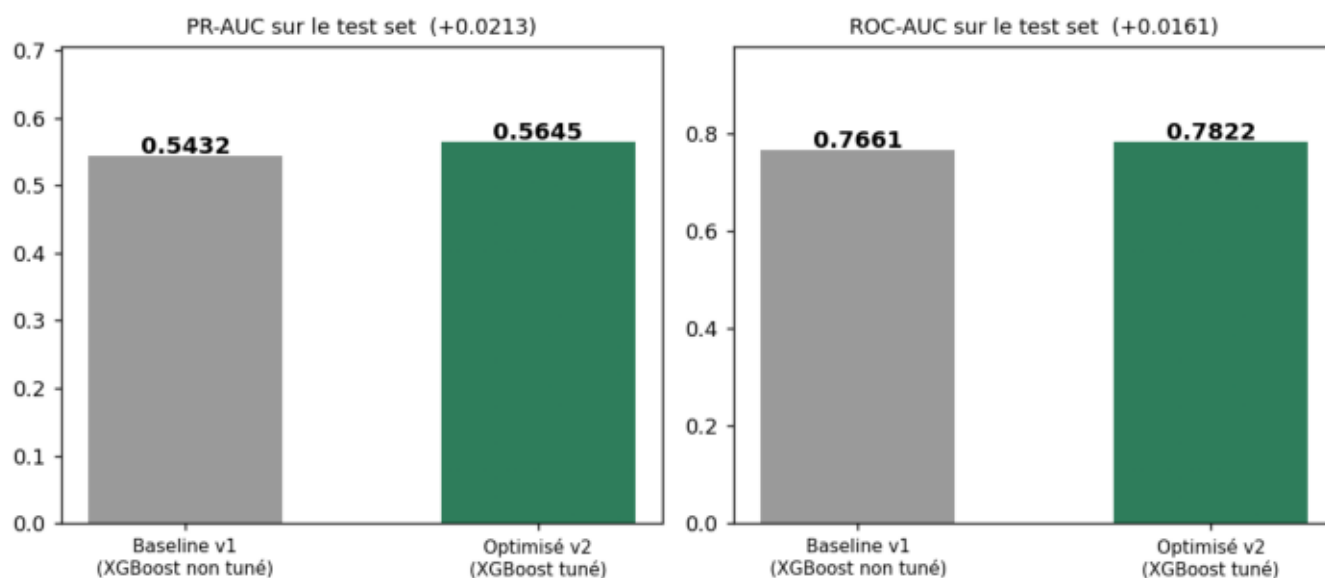
Probabilité prédite vs taux de défaut réellement observé (out-of-fold, train)

Le plafond de discrimination n'empêche pas le modèle d'être utile ni bien calibré : il sépare correctement un groupe à 4.5% de risque d'un groupe à 70.2% de risque, soit un rapport de 15x, directement exploitable pour prioriser des actions de recouvrement.

Mais même dans le groupe le plus risqué, 30% des clients ne font pas défaut. Ce n'est pas une erreur du modèle : ces clients sont strictement identiques aux défaillants sur toutes les variables disponibles. Pour les distinguer, il faudrait d'autres DONNÉES (revenus, historique bancaire, encours dans d'autres établissements), pas d'autres algorithmes.

16. Évaluation finale sur le test set

6 000 clients, jamais utilisés avant



Métrique	Version 1	Version 2	Gain	Gain relatif
PR-AUC	0.5432	0.5645	+0.0213	+3.9%
ROC-AUC	0.7661	0.7822	+0.0161	+2.1%

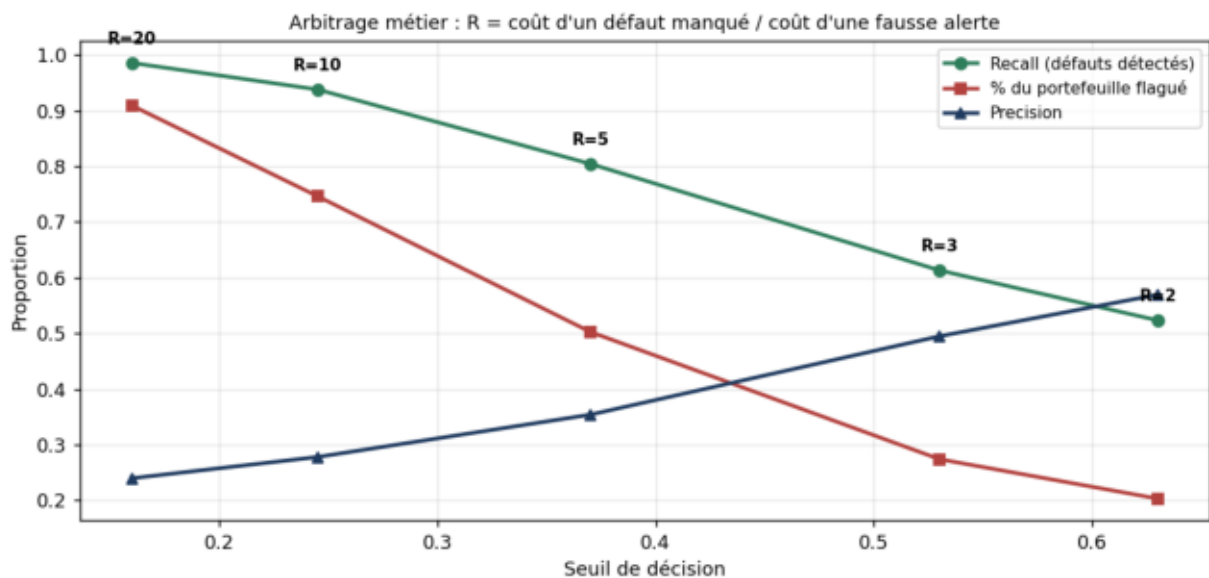
Le gain mesuré en validation croisée se confirme sur des données jamais vues, ce qui exclut un artefact de sélection. Il provient entièrement du tuning d'hyperparamètres— ni du feature engineering (section 9), ni du rééquilibrage (déjà optimal en partie 1), ni du deep learning (section 13), ni du blending (section 14).

Modèle final retenu : XGBoost avec `learning_rate = 0.010`, `max_depth = 4`, `gamma = 4.3`, `min_child_weight = 9`, 517 arbres, `scale_pos_weight = 3.521`.

17. Choix du seuil par coût métier

La version 1 maximisait le F2, ce qui donnait un seuil de 0.258 flaguant 60% du portefeuille— inexploitable. On raisonne désormais en coût explicite : coût total = $R \times (\text{défauts manqués}) + (\text{fausses alertes})$. Seul le RAPPORT R importe, la banque n'a donc pas besoin de chiffrer ses coûts en valeur absolue.

R = coût FN / FP	Seuil	Recall	Precision	% portefeuille flagué
2	0.630	52.3%	57.0%	20.3%
3	0.530	61.3%	49.5%	27.4%
5	0.370	80.4%	35.4%	50.3%
10	0.245	93.8%	27.8%	74.7%
20	0.160	98.6%	24.0%	90.9%



Seuils calibrés sur les probabilités out-of-fold du train, puis appliqués une seule fois au test. C'est cette table qui doit être présentée au métier pour arbitrer : si un défaut manqué coûte 3x une fausse alerte, le seuil est 0.530 et l'on flague 27% du portefeuille en détectant 60% des défauts. S'il coûte 10x, le seuil descend à 0.245, on détecte 94% des défauts mais on flague 76% des clients. Le choix appartient à la banque, pas au modèle.

18. Synthèse et recommandations

- Ce qui a produit un gain : le tuning d'hyperparamètres, et lui seul (+0.0213 de PR-AUC sur le test, soit +3.9%). Les paramètres initiaux sur-apprenaient.
- Ce qui n'a rien produit : le feature engineering (4 jeux testés, écarts sous le bruit), le deep learning séquentiel (LSTM/GRU au niveau d'un simple MLP, sous le boosting), et le blending (+0.0014 sur 682 combinaisons testées).
- Ce qui était déjà bon : la stratégie de rééquilibrage. `class_weight` / `scale_pos_weight` reste le meilleur choix — aucune donnée dupliquée, aucun identifiant client inventé.
- Le plafond est dans les données : démontré par trois tests indépendants — corrélation inter-modèles de 0.905, blending sans effet, et convergence de deux boostings distincts sur la même valeur au millième.
- Le modèle reste utile : il distingue un groupe à 4.5% de risque d'un groupe à 70%, soit un rapport de 15x, et il est bien calibré — donc directement exploitable pour prioriser des actions.
- Pour progresser davantage : il faut de nouvelles DONNÉES (revenus, ancienneté bancaire, encours dans d'autres établissements, incidents externes), pas de nouveaux algorithmes. C'est la recommandation principale de ce rapport.
- Seuil de décision : à arbitrer avec le métier via la table de coûts (section 17), jamais par une formule automatique.

Prochaine étape — industrialisation

Le code est déjà structuré pour cela : `src/data_prep.py` (chargement, nettoyage, features), `scripts/run_experiments.py` (expériences reproductibles avec reprise après interruption), `scripts/final_eval.py` (évaluation et seuils). Reste à ajouter un script d'entraînement produisant un modèle sérialisé, une API d'inférence, et un suivi de dérive des données en production.